TECHNICAL REFERENCE MANUAL

Live Sports Match Data Aggregator

Complete Class, Service & Controller Specification

Scope: Exhaustive architectural mapping of all 34 Java classes
Alignment: 1-to-1 fulfillment of the FanCode engineering brief
Architecture: Controller → Service → Repository → Entity
Verification: 11/11 Automated Tests Passing

Author: Deepak Agrawal • Date: November 2026
Repository: `C:\Users\Deepak\OneDrive\Desktop\fancode-aggregator`

1. Requirement Traceability Matrix

This matrix maps every specific requirement from the FanCode prompt to the exact Java classes and methods that satisfy it:

Requirement from Specification	Implementing Class(es)	Key Method(s)	Invariant Enforced
Push Webhook Ingestion	`WebhookController` `AggregatorService`	`handleProviderAWebhook()` `processPushEvent()`	Accepts JSON payload with `deliveryId`, `occurredAt`, `snapshot`, and `commentary`.
Delivery Idempotency	`ProviderDeliveryEntity` `ProviderDeliveryRepository`	`existsByProviderAndDeliveryId()`	Duplicate `(provider, deliveryId)` returns `duplicate_ignored` with zero database modifications.
Snapshot Strict Monotonicity	`ProviderSnapshotEntity` `AggregatorService`	`isCandidateStrictlyNewer()` `applySnapshotInternal()`	Lexicographical check: `(5,1) > (4,9)`, `(4,2) > (4,1)`. Older snapshots rejected without throwing.
Commentary Sequence Ordering	`ProviderCommentaryEntity` `ProviderCommentaryRepository`	`findByMatchIdAndProviderOrderBySequenceAsc()`	Balls ordered strictly by provider `sequence`, not arrival time. Missing sequence gaps are supported.
Commentary Revisions / Corrections	`ProviderCommentaryEntity` `AggregatorService`	`applyCommentaryInternal()`	Higher revision replaces text for stable `ballId`. Equal/lower revision ignored. Exactly 1 record per ball.
Deterministic Provider Precedence	`AppProperties` `AggregatorService`	`determineActiveSnapshot()` `determineActiveProvider()`	Primary provider (`provider-a`) prioritized; falls back to secondary (`provider-b`). Sequences never compared.
Polling Provider B with 503 Resilience	`ProviderBPollerService` `MockProviderBService`	`pollOnce()` `scheduledPoll()`	Catches HTTP 503 and preserves last good state. Repeated identical polls create no duplicate balls.
Push Worked Example Generator	`ProviderAGeneratorService` `MockProviderAController`	`runWorkedPushScenario()` `playWorkedScenario()`	Deterministic execution of the 6-step prompt timeline (d1, d2, d2 duplicate, d4, d3 late, d5 correction).

Requirement from Specification	Implementing Class(es)	Key Method(s)	Invariant Enforced
Client Read APIs	`ClientApiController`	`GET /matches`, `GET /score`, `GET /commentary`	Returns normalized views. Unknown match returns structured HTTP 404 with error payload.
Durable Storage Across Restarts	`application.yml` All JPA Entities	`jdbc:h2:file:./data/fancodedb;DB_CLOSE_DELAY=-1`	All matches, scores, and commentary survive JVM restarts and system reboots.

2. Controllers Layer (`com.fancode.aggregator.controller`)

2.1 `ClientApiController`

Serves all read queries requested by client applications. Operates strictly in read-only mode (`@Transactional(readOnly = true)`).

- `GET /matches` : Iterates through all matches in the database. Resolves the active provider snapshot for each and returns a compact summary list (e.g. `[{"id": "match-1001", "status": "live", "summaryScore": "10/1 (1.1)"}]`).
- `GET /matches/{id}` : Retrieves full details of a specific match, including canonical ID, current status, active provider, and complete scoreboard object. Throws `ResourceNotFoundException` if match does not exist.
- `GET /matches/{id}/score` : Returns the authoritative scoreboard object: status, innings, runs, wickets, overs string, and formatted score (`"10/1"`).
- `GET /matches/{id}/commentary` : Returns all commentary balls from the active provider, sorted chronologically by sequence number.

2.2 `WebhookController`

The ingestion entry point for Provider A push webhook events.

- `POST /api/webhooks/provider-a` : Accepts a `PushEventRequest` body. Delegates to `AggregatorService.processPushEvent(...)` and returns a `PushEventResponse` with HTTP 200.

2.3 `MockProviderAController`

Exposes an operational trigger to run the exact worked push scenario without needing an external script.

- `POST /mock/provider-a/play` : Invokes `ProviderAGeneratorService.runWorkedPushScenario(matchId)` and returns the array of 6 step execution responses.

2.4 `MockProviderBController`

Simulates Provider B's polling endpoints as required by the assignment brief.

- `GET /provider/matches/{id}/score` : Emits the current Provider B score snapshot.
- `GET /provider/matches/{id}/commentary` : Emits the full array of commentary items.
- `POST /mock/provider-b/scenario/{name}` : Alters the mock state machine (`initial` , `unchanged` , `newer_score` , `new_ball` , `stale` , `error_503`).
- `POST /mock/provider-b/poll` : Triggers an on-demand poll cycle immediately.

3. Core Services Layer (`com.fancode.aggregator.service`)

3.1 `AggregatorService`

The core domain engine. Handles all ordering, idempotency, revision updates, and provider precedence.

- `processPushEvent(PushEventRequest event)` : Synchronizes on an ingestion lock. Evaluates `deliveryRepository.existsByProviderAndDeliveryId()` . If duplicate, returns `duplicate_ignored` . Otherwise, records delivery ID, resolves match ID, and calls internal snapshot and commentary processors.
- `applySnapshotInternal(...)` : Compares incoming snapshot `(seq, rev)` against stored snapshot using `ProviderSnapshotEntity.isCandidateStrictlyNewer()` . If strictly newer, updates the provider snapshot row and updates match precedence. If older or equal, skips update.
- `applyCommentaryInternal(...)` : Looks up commentary by `(matchId, provider, ballId)` . If ball exists and `candidate.rev > stored.rev` , updates text in-place. If ball does not exist, inserts a new entity. Equal or lower revisions are discarded.
- `determineActiveSnapshot(matchId)` & `determineActiveProvider(matchId)` : Implements primary provider precedence (`provider-a` priority, fallback to `provider-b`). **Never compares sequence numbers across providers.**
- `getMatchCommentary(matchId)` : Queries `commentaryRepository.findByMatchIdAndProviderOrderBySequenceAsc()` . Guarantees that late balls arriving out of order are displayed in their proper sequential order.

3.2 `ProviderAGeneratorService`

Fulfills the requirement that the push generator must deterministically reproduce all 6 steps from the prompt: Step 1 (d1 initial), Step 2 (d2 out), Step 3 (d2 duplicate), Step 4 (d4 gap arrival), Step 5 (d3 late older snapshot), and Step 6 (d5 revision 2 correction).

3.3 `ProviderBPollerService`

Manages scheduled background polling (via Spring `@Scheduled`) and on-demand polling. Wraps calls in try-catch to intercept `MockProviderUnavailableException` (503 Service Unavailable), logging the outage while leaving stored match data untouched.

3.4 `MockProviderBService`

Configurable in-memory state machine implementing Provider B's score and commentary data feeds with full support for unchanged responses, new balls, newer scores, stale tuples, and HTTP 503 errors.

4. Entities, Repositories & DTOs

Entities (`com.fancode.aggregator.entity`)

- **`MatchEntity`**: Represents the match known to clients (`match-1001`). Stores `status` and `active_provider` .
- **`ProviderDeliveryEntity`**: Stores `provider` and `deliveryId` with a compound unique key for push message deduplication.
- **`ProviderSnapshotEntity`**: Stores the latest scoreboard tuple `(sequence, revision, runs, wickets, overs)` per provider. Compound unique constraint on `(match_id, provider)` .
- **`ProviderCommentaryEntity`**: Stores individual balls with stable `ballId` , display `sequence` , and `revision` . Indexed on `(match_id, provider, sequence_num)` for fast chronological retrieval.

Repositories (`com.fancode.aggregator.repository`)

Spring Data JPA interfaces generating type-safe SQL queries automatically:

- `MatchRepository` : CRUD operations on canonical matches.
- `ProviderDeliveryRepository` : Fast existence queries via `existsByProviderAndDeliveryId()` .
- `ProviderSnapshotRepository` : Partitioned scoreboard queries via `findByMatchIdAndProvider()` .
- `ProviderCommentaryRepository` : Ball lookup via `findByMatchIdAndProviderAndBallId()` and ordered queries via `findByMatchIdAndProviderOrderBySequenceAsc()` .

Data Transfer Objects (`com.fancode.aggregator.dto`)

Implemented as immutable Java 21 `record` types with Jackson JSON annotations:

- `PushEventRequest` / `PushEventResponse` : Webhook wire protocol.
- `SnapshotDto` / `CommentaryDto` : Internal transport objects for scoreboard and commentary items.
- `PollScoreResponse` / `PollCommentaryResponse` : Wire payloads for Provider B endpoints.
- `MatchSummaryResponse` , `MatchDetailResponse` , `ClientScoreResponse` , `ClientCommentaryItemResponse` : Normalized client read representations.
- `ErrorResponse` : RFC-compliant structured error object.

Exceptions & Handlers

- **`ResourceNotFoundException`**: Unchecked exception thrown when querying non-existent match IDs.
- **`GlobalExceptionHandler`**: Annotated with `@RestControllerAdvice` . Intercepts `ResourceNotFoundException` and returns HTTP 404 with structured JSON.

5. Test Suite Verification & Mapping

Test Class	Test Method	Verification Details	Status
`AggregatorIntegrationTests`	`testOrderedIngestion`	Ingests d1 (4/0) then d2 (4/1). Verifies sequential state progression.	PASS
`AggregatorIntegrationTests`	`testDuplicatePushDelivery`	Sends d1 twice. Verifies second returns `duplicate_ignored` and DB state is unchanged.	PASS
`AggregatorIntegrationTests`	`testOlderSnapshotWithLateBallReturnedInSequence`	d4 arrives before d3. Verifies score 10/1 remains active when d3 arrives, while ball b3 is inserted in sequence order.	PASS
`AggregatorIntegrationTests`	`testCommentaryCorrection`	Ball b2 rev 2 replaces text with "out, review stays out". Sending rev 1 later is ignored.	PASS
`AggregatorIntegrationTests`	`testRepeatedIdenticalPolls`	Polls Provider B 3 times in unchanged state. Exactly 1 ball exists in DB.	PASS
`AggregatorIntegrationTests`	`testStalePoll`	Polls sequence 16 (104/2), then stale sequence 10 (80/1). Score remains 104/2.	PASS
`AggregatorIntegrationTests`	`testProviderErrorLeavesLastGoodState`	Simulates HTTP 503 on Provider B. Poller handles cleanly; last good score is preserved.	PASS
`AggregatorIntegrationTests`	`testFullWorkedPushScenario`	Executes full 6-step prompt timeline. Final score is 10/1, commentary has all 4 balls.	PASS
`ClientApiControllerTest`	`testUnknownMatchReturns404`	Verifies unknown match ID returns HTTP 404 with error JSON.	PASS
`ClientApiControllerTest`	`testWebhookAndClientReadFlow`	Full end-to-end webhook POST and client read query verification.	PASS
`DatabaseRestartPersistenceTest`	`testRestartAgainstSameDatabase`	Boots Spring Context 1, writes data, closes it, boots Context 2 against disk file, verifies 100% data recovery.	PASS

> **Summary:** All 34 Java classes cleanly fulfill their architectural responsibilities. The application builds, passes all 11 unit & integration tests with 0 failures, and is fully packaged as a standalone runnable JAR and Docker container.